

High Performance and Distributed Computing for Big Data

Unit 3: Cloud Systems and Big Data Management

Session 6 & 7 - AWS Networking

Francesc Solsona francesc.solsona@udl.cat

Universitat Rovira i Virgili and Universitat de Lleida

Recap

Networking concepts we already met

Throughout the course, we have already been interacting with different networking concepts. Here are some of them:

- We have been **connecting to** EC2 instances using SSH.
- We have been using the **public IP** address of the EC2 instances.
- We have been using the **DNS** name of the EC2 instances.
- We have been using different **ports** to launch different services

Although we have been working with these concepts, we haven't really discussed them in detail. Here are some questions that we can ask ourselves:

- Why do we sometimes specify **which port to use** when connecting to a service?
- Why do we use the public IP address of an EC2 instance instead of the private IP address?
- Why can we use the DNS name and the public IP address **interchangeably**?
- What is the **difference between a public and a private** IP address?
- Why is the **IP address changing** every time we restart the lab?

Networking basics

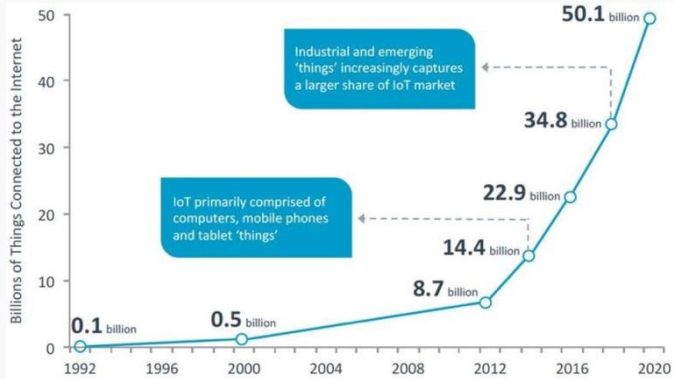
What is an IP address?

An **IP address** is a unique identifier for a device. The same way a street address identifies a house, an IP address identifies a device on a network.

- An IP address is a 32-bit number that is divided into four 8-bit numbers separated by dots. For example, `1.2.3.4` or `52.4.75.36`.
- Every one of these 8-bit numbers is called an **octet** and can have a value between 0 and 255.
- All IP addresses in the world have a value between 0.0.0.0 and 255.255.255.255 which makes up a total of 4,294,967,296 possible IP addresses (2^{32}).

So we have around four billion IP addresses. This might seem like a lot, but **it is not enough to give an IP address to every device in the world.**

What is an IP address?



How do we solve this? Any idea?

Private vs Public IP addresses

To solve the problem of not having enough IP addresses, we use **private** and **public** IP addresses.

- **Private IP addresses** are used to identify devices within a network. They are not unique and can be reused in different networks.
- **Public IP addresses** are used to identify devices on the internet. They are unique and can't be reused.
- **Address Ranges** are reserved for private and public IP addresses. IPs that fall into `10.x.x.x, 172.16.x.x - 172.31.x.x, 192.168.x.x` are reserved for private IP addresses.

Think of it as a hotel. The room number is the **private IP address** and the hotel street address is the **public IP address**.

- The room number is unique within the hotel, but not unique across all hotels.
- The hotel street address is unique and can't be reused.
- The hotel street address is used to identify the hotel from the outside world, while the room number is used to identify the room within the hotel.

The **Domain Name System (DNS)** is a system that translates domain names to IP addresses. Remember during class when I said you can use the DNS name or the public IP address to connect to an EC2 instance? A DNS name or a domain name is a human-readable name that references an IP address.

If we go back to the hotel analogy, the hotel name is the **DNS name** and the hotel street address is the **public IP address**.

To know to which IP address a domain name corresponds, we use a **DNS server**. This server is like a phone book that translates domain names to IP addresses.

Ports

Once we know the address of a device, whether we already knew the IP address or the DNS name, we need to know **which service** we want to connect to. So imagine the EC2 machine we configured with jupyter notebook. That machine has at least two services running: the SSH service (port 22) and the Jupyter notebook service (we used port 8888).

When we connect to the EC2 machine through SSH, we don't need to specify the port because the default port for SSH is 22, but we could if we wanted.

```
$ ssh -i ~/.ssh/aws-keypair ec2-user@<your-ec2-instance-public-ip> -p 22
```

The above command will work the same as if we didn't specify the port. But notice that if we specify a different port:

```
$ ssh -i ~/.ssh/aws-keypair ec2-user@<your-ec2-instance-public-ip> -p 1234
```

We get an error:

```
ssh: connect to host <your-ec2-instance-public-ip> port 1234: Connection timed out
```

What did we do to access the jupyter notebook service? We used the public IP address of the EC2 instance (or the DNS name) and the port 8888. But instead of using the SSH command, we typed the following in the browser:

```
http://<your-ec2-instance-public-ip>:8888
```

That is because for SSH, we use the `ssh` command and point to a specific IP address (or the DNS name) and port. But for jupyter notebooks, we use the browser and point to a specific IP address and port.

In both cases we are accessing a service on a remote device.

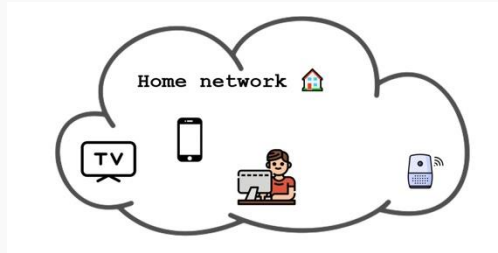
We've seen how IP addresses are used to **identify devices**, and how there's private and public IP addresses that are used to identify devices **within a network** and **on the internet** respectively. Some questions to think about:

- How do multiple devices in a network share a single public IP address?
- What even is a network?
- How do we manage the IP addresses in a network? How do we choose which devices get which IP addresses?

Let's explore these questions further starting with a situation you may be familiar with: your home network.

Our home network

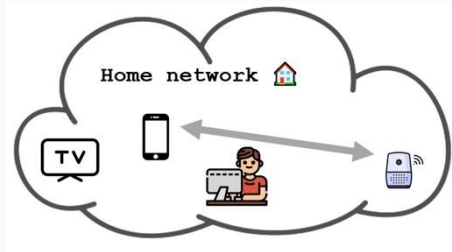
Let's imagine a simple home network where we have some devices connected. So for example we could have the computer we are using right now, a phone, a smart TV and a smart speaker (like an Amazon Echo).



Our home network

The devices can talk between themselves. For example, when I am playing a song on spotify on my phone, I can send it to the smart speaker so my smart speaker is now playing the song.

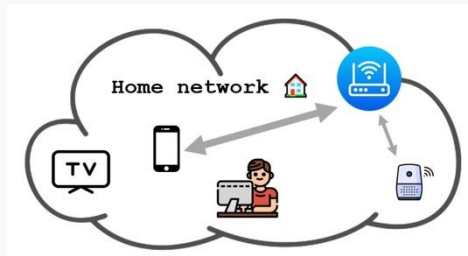
But what is happening? Is my phone directly talking to my smart speaker?



Your router and NAT

No! There is a key player and I am sure you have heard of it: the **router**. The device we must restart when the internet is not working.

The router is the **receptionist** of the network. It is the one responsible for routing traffic between devices in the network and between the network and the internet. So if my phone wants to talk to the smart speaker, it sends the message to the router and the router sends it to the smart speaker.



Your router and NAT

Remember we said there are different private IP ranges reserved for private networks? Well, the router uses these private IP addresses to identify the devices in the network. So when my phone sends a message to the smart speaker, **it uses the private IP address of the smart speaker**. The same way a hotel uses room numbers to identify rooms.

In fact, let's do an experiment. Open a terminal and run the following command if you are on Windows:

```
Node> ipconfig
```

Or if you are on Linux or MacOS:

```
$ ifconfig
```

If you look for your `IPv4 Address`, you will see an address that fits into 192.168.x.x, 10.x.x.x or 172.16.x.x - 172.31.x.x. Most of the times on regular homes it will be 192.168.x.x. This is analogous to how different hotels will use the same room numbers (like room 101) that **are not unique in the world**.

If you instead visit a website like whatismyip.com, you will see a different IP address. **This is the public IP address of your network**. This is the address that identifies your network to the **outside world**.

Let's see now how the router uses the **public IP address to communicate with the internet**.

Imagine you are staying in the hotel and want to send a letter to a friend's home (and imagine the hotel is willing to send the letter for you). You would write the address of your friend's home on the envelope, hand it to the receptionist, and the receptionist would send it to your friend's home.

More people on the hotel may be sending letters to their respective friends' homes, so the receptionist **needs to keep track of which letter goes to which home** so when the responses come back, the receptionist knows to whom to give the letter.

Your router and NAT

The router does the same thing. If your phone is streaming a YouTube video, and your smart TV is streaming a Netflix movie, the router is handling traffic between your phone and YouTube and between your smart TV and Netflix **both through the same public IP address** and the **public IPs of YouTube and Netflix**.

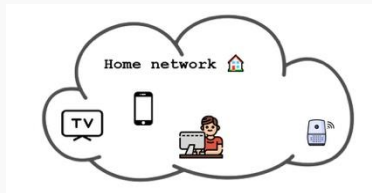
The router keeps track of which traffic is going where so when the responses come back, it knows to whom to send the response. In networking terms, this is called **Network Address Translation (NAT)** and is the protocol routers use to allow multiple devices with private IP addresses to share a single public IP address. Read more about NAT [here](#).

What is a network?

So far we have been talking about networks, using private IPs to communicate between devices within the network and public IPs to communicate with the internet. But **what is a network?** And **what is the internet?**

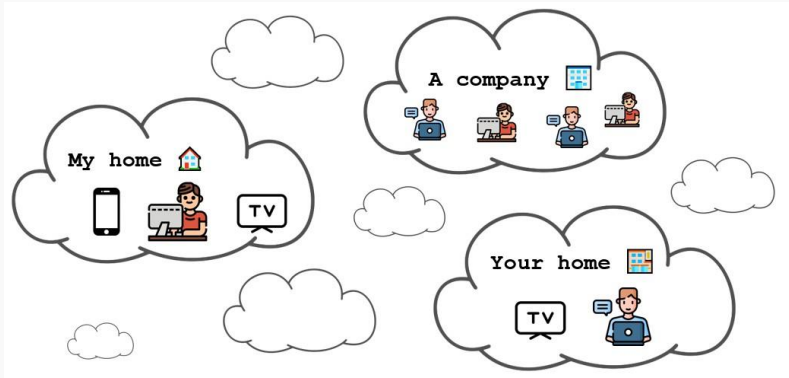
A network is a collection of devices that are connected to each other. So in our home network example, the phone, the smart TV, the smart speaker, and the computer are all part of the network and they are identified by **private IPs**.

In the hotel analogy, the network is the hotel and the different devices are the rooms, where the room number would be the private IP address.



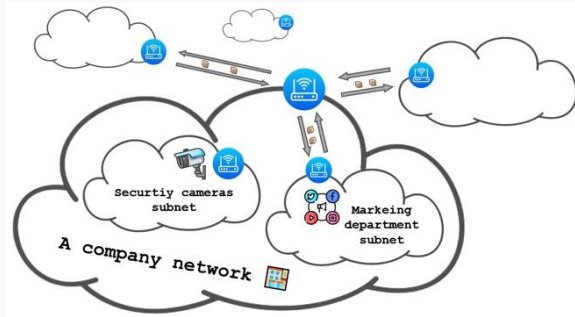
What is a network?

But our home network is not the only network in the world. There are many networks all over the world; your home network, a given company network, the network of a university, etc.



What is the internet?

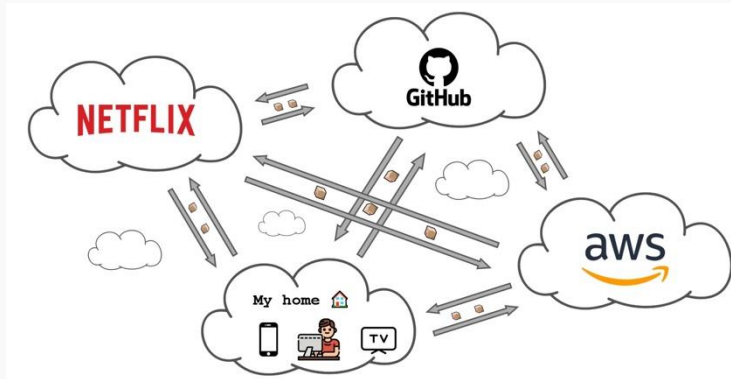
Networks are **hierarchical**. So the network of a company can be made up of different networks, like the network of the sales department, the network of the marketing department, etc.



What is the internet?

Although private networks exist, the most common scenario is for networks to be connected to the rest of the world. In fact, the **internet** is nothing more than the biggest network of networks in the world.

That is why from our home network we can access anything available on the internet, like YouTube, Netflix, AWS or even this lecture. The whole picture would look like this:



What is the internet?

We constantly use concepts like *the cloud* or *the internet* without really thinking that this is just a colossal amount of devices connected between them.

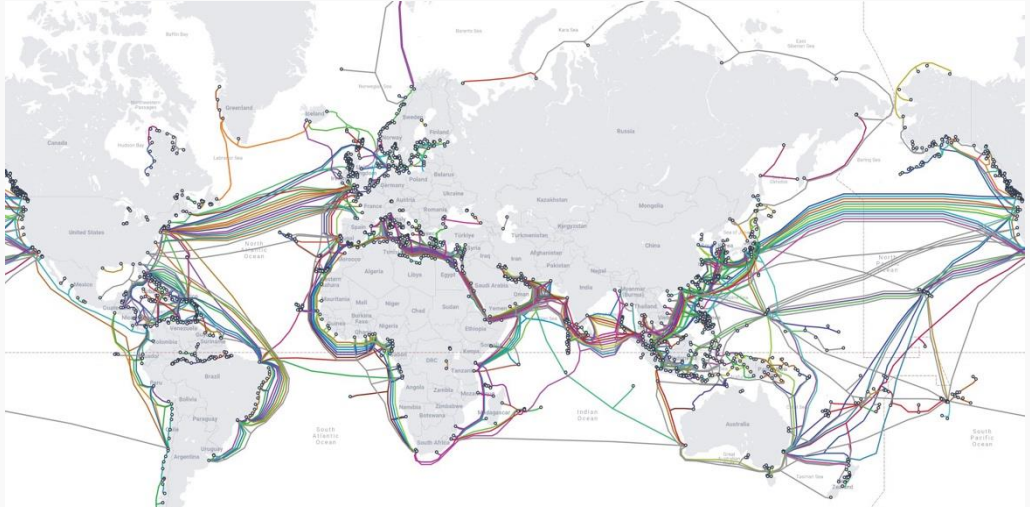
When I want to connect to an EC2 machine in *the cloud*, (assuming it is on the default region `us-east-1`), I am just sending electrical signals to a computer located in a datacenter in the United States.

What is the internet?

Think for a moment the insane trip that the electrical signals have to make to go from your computer to the EC2 machine. They will first go from my computer to my router, from my router at my home to a series of routers managed by Internet Service Providers (ISPs) in Spain, from there it has to **cross the Atlantic Ocean** to reach the United States, and then go through a series of routers managed by ISPs in the United States to finally reach the datacenter where the EC2 machine is located.

The internet is possible because of an immense infrastructure of cables, routers, switches, and servers that are all connected to each other. In fact, there are a huge amount of cables that go under the ocean to connect the different continents. You can see a map of these cables submarinecablemap.com.

What is the internet?



What is the internet?

Have you ever thought that when you are doing anything on the internet, you may be sending electrical signals that are **crossing the ocean**?



We can do a quick experiment to know how far the electrical signals have to travel to reach a given location. We can use the `ping` command to measure the time it takes for a signal to go from our computer to a given location and back.

```
$ ping google.es
```

The above command will send a signal to google.es and measure the time it takes for the signal to go and come back. The time it takes is called the **round-trip time** and is measured in milliseconds. The round-trip time is a good indicator of how far the location is from your computer. The bigger the round-trip time, the farther the location is.

To see how much time does it take for network traffic to reach different parts of the world, we can use domain names that we know are located in different countries. For example, we can use the following domains:

```
$ ping es.pool.ntp.org # Spain  
$ ping us.pool.ntp.org # United States  
$ ping ru.pool.ntp.org # Russia  
$ ping nz.pool.ntp.org # New Zealand
```

Another command we can use to explore the internet is the `tracert` command (or `tracert` for **MacOS or Linux users**). This command shows the path that the network traffic takes to reach a given location by printing the IP addresses of the routers that the traffic goes through.

```
$ tracert es.pool.ntp.org # Spain
$ tracert us.pool.ntp.org # United States
$ tracert ru.pool.ntp.org # Russia
$ tracert nz.pool.ntp.org # New Zealand
```

We've seen how a network is a collection of devices that are connected to each other and how the internet is the biggest network of networks in the world. Some questions to think about:

- How can a single network be made up of multiple networks?
- How does my router choose which IP address to give to the devices that get connected?
- How does a router decide which traffic goes where and which subnet can talk to which other subnet?

Subnetting

What is a subnet?

While explaining what is the internet, we mentioned that a network can be made up of different networks.

This is where **subnets** come into play. A **subnet** is a way of dividing a network into smaller networks.

The subnets allow us to divide a network into smaller networks and control which subnet can talk to which other subnet.

In the case of a company we may want the subnet of sales department and the subnet of the marketing department to be able to talk to the subnet of IT administration, but we don't want the subnet of sales to be able to talk to the subnet of marketing.

We could also have a security camera subnet that we don't want to communicate with any other subnet **nor with the internet**.

What is a subnet?

Remember the private IP address ranges we mentioned earlier?

10.x.x.x, 172.16.x.x – 172.31.x.x, 192.168.x.x.

How do we define address ranges for subnets?

You may have noticed that IP addresses often come with a trailing slash and a number. For example, 192.168.1.0/24.

This /24 is called the **subnet mask** and it means that we are not in front of an IP address but an **IP address range**, in this case, the range of IP addresses between 192.168.1.0 and 192.168.1.255.

What is a subnet?

Remember we earlier said that an IP address is a 32-bit number divided into four 8-bit numbers **separated by dots**. Which in simple terms just means that an IP address is built from 4 numbers between 0 and 255 separated by dots.

The subnet mask is a way of telling **which part of the IP address is fixed and which part can change**. In the case of `/24`, the first 24 bits are fixed and the last 8 bits can change, which means that the last number in the IP address can be any number between 0 and 255.

Other common subnet masks are `/16` and `/8`. `/16` means the first 16 bits are fixed and the last 16 bits can change. `/8` means the first 8 bits are fixed and the last 24 bits can change.

What is a subnet?

Depending on how many octets of the IP address are fixed, we can have a different amount of private IPs available.

Subnet Mask	Number of IPs
/8	16,777,216 (2^{24})
/16	65,536 (2^{16})
/24	256 (2^8)

What is a subnet?

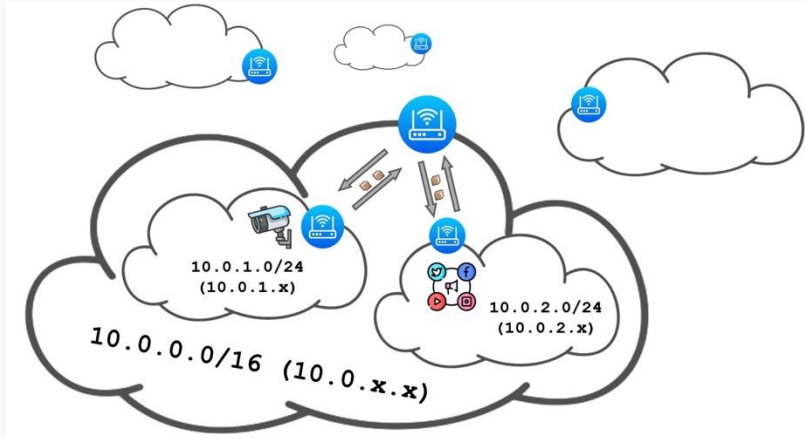
Let's see an example of how we can divide a network into subnets. Imagine we have a network with the IP address range `10.0.0.0/16`. This means **in total we have 65,536 IP addresses** available ranging from `10.0.0.0` to `10.0.255.255`.

If we wanted to divide this network into two subnets, we could use the following IP address ranges:

- Subnet 1: `10.0.1.0/24` (IP addresses from `10.0.1.0` to `10.0.1.255`)
- Subnet 2: `10.0.2.0/24` (IP addresses from `10.0.2.0` to `10.0.2.255`)
- Subnet 3: `10.0.3.0/24` (IP addresses from `10.0.3.0` to `10.0.3.255`)
- etc.

What is a subnet?

So if we had a company where we want to divide our network into the security cameras subnet and the marketing department subnet, we could have the following setup:



What is a subnet?

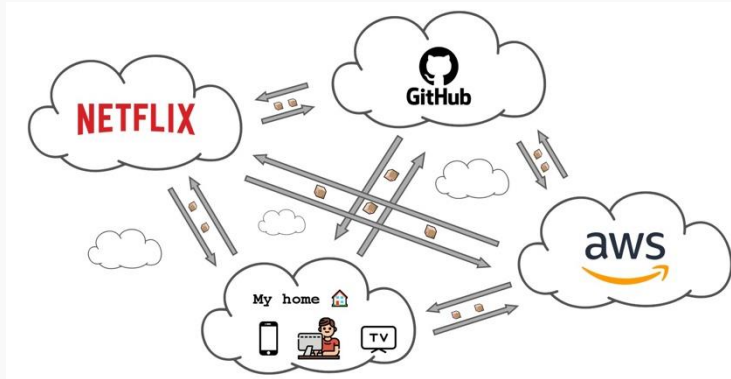
And how do routers apply different rules to different subnets? They use **route tables**. It consists of a list of rules that specify which traffic can go where.

We could build a route table that says that traffic from the marketing department and security cameras can go to the internet (so we may simplify and say that traffic originating from **any** subnet can go to the internet), but traffic from the marketing department can't go to the security cameras subnet and vice versa.

Networking in AWS

How do AWS networks work?

So far we have been talking about networks in general, but how do networks work in AWS? Just as a reminder, remember we are in this **network of networks called the internet**.



How do AWS networks work?

And of course, AWS is part of the internet.



How do AWS networks work?

But what happens inside the AWS network? How do **different users** have its own individual space?

Each user in AWS has its own set of networks. These networks are called **Virtual Private Clouds (VPCs)** which is just a fancy name for the AWS service that lets us create and manage our own networks inside AWS.

By default, everything we launch in AWS lives **inside the same VPC**, which is one that AWS creates and configures for us. But we can create other VPCs and configure them as we want.

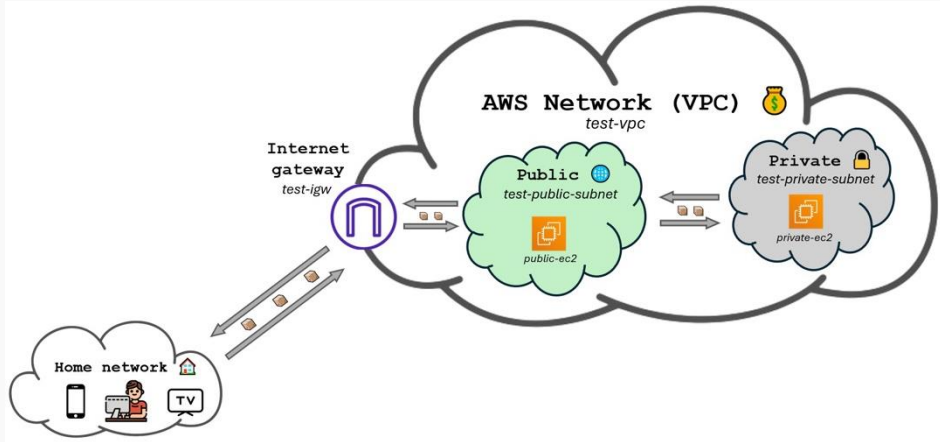
Lab: Experimenting with VPCs

Goal of the lab

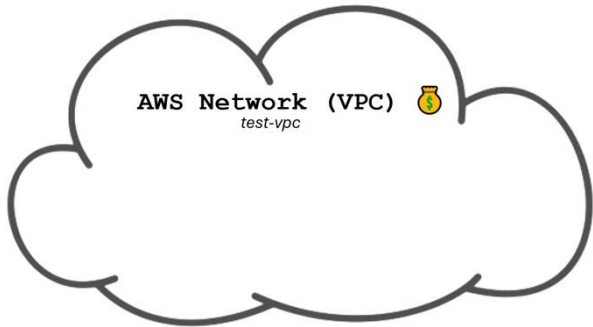
During this lab we will:

- Create a **VPC**.
- Create two **subnets** in the VPC.
- Configure one subnet as **public** and the other as **private** with an **Internet Gateway** and **Route Tables**.
- Deploy an **EC2 instance** in both subnets.
- Access the private EC2 instance from the public EC2 instance.

Goal of the lab



Creating a VPC

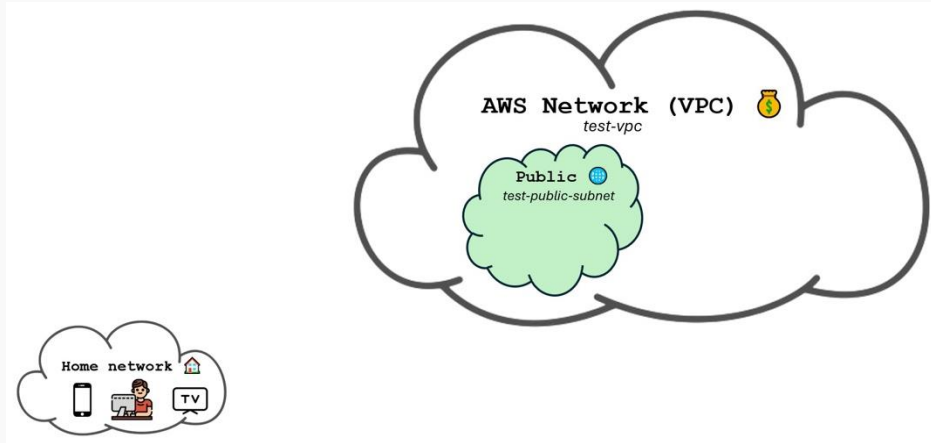


Creating a VPC

1. Go to the VPC service.
2. Click on Create VPC.
3. Choose ☐ VPC only.
4. Fill the form with the following settings:
 - **Name tag:** test-vpc
 - **IPv4 CIDR block:** 10.0.0.0/16
5. Leave the rest as default and click on Create VPC.

Creating a public subnet in a VPC

We are now going to add a subnet to the VPC we just created.

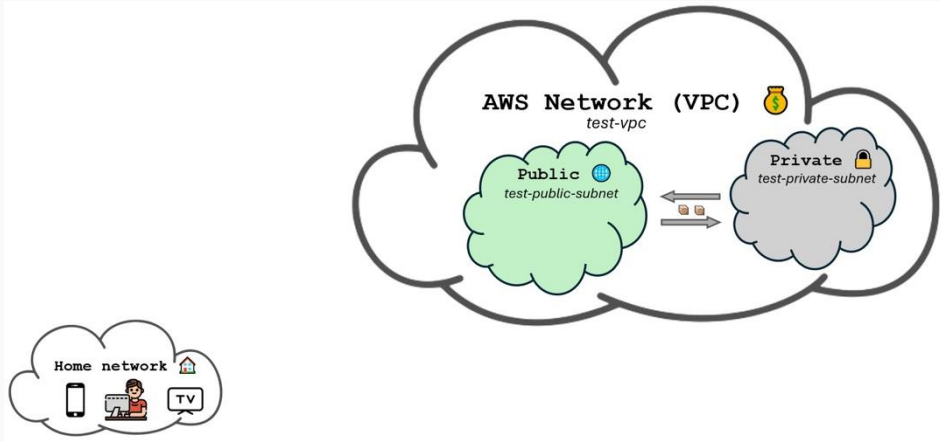


Creating a public subnet in a VPC

1. Navigate to the *VPC service* in the *AWS Management Console*.
2. Select **Subnets** on the left navigation pane.
3. Click on *Create subnet*.
4. Select the VPC you created, which is the one ending with (test-vpc).
5. Fill in the form with the following settings:
 - **Name:** test-public-subnet
 - **Availability Zone:** Select us-east-1a zone (or any other zone but be consistent)
 - **IPv4 VPC CIDR block:** 10.0.0.0/16 (leave as default)
 - **IPv4 subnet CIDR block:** 10.0.1.0/24
6. Click on *Create subnet*.

Creating a private subnet in a VPC

And we add a second one.

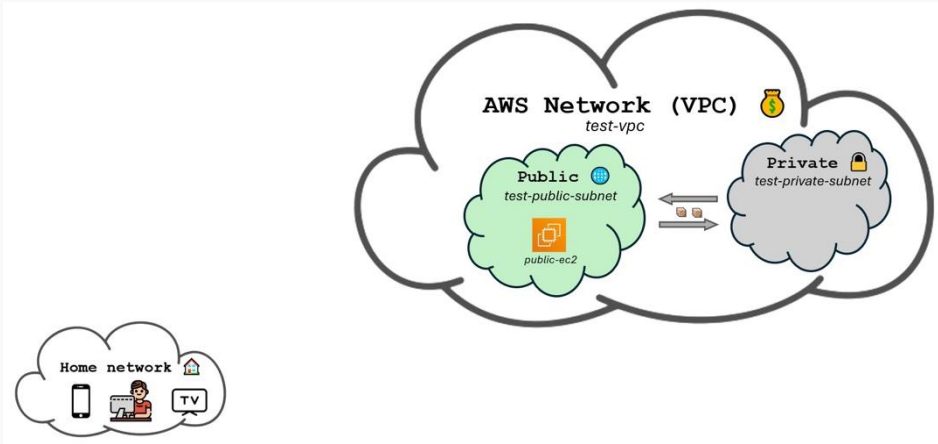


Creating a private subnet in a VPC

1. Navigate to the *VPC service* in the *AWS Management Console*.
2. Select **Subnets** on the left navigation pane.
3. Click on *Create subnet*.
4. Select the VPC you created, which is the one ending with (test-vpc).
5. Fill in the form with the following settings:
 - **Name:** test-private-subnet
 - **Availability Zone:** Select us-east-1a zone (the same as the public subnet)
 - **IPv4 VPC CIDR block:** 10.0.0.0/16 (leave as default)
 - **IPv4 subnet CIDR block:** 10.0.2.0/24
6. Click on *Create subnet*.

Deploying an EC2 instance in the public subnet

Now we are going to launch an EC2 instance inside the public subnet.



Deploying an EC2 instance in the public subnet

1. Go to the *EC2 service*.
2. Click on *Launch instance*.
3. Fill the form with the following settings:
 - **Name:** public-ec2
 - **Key pair (login):** Choose `aws-keypair` (the one we have already been using)
4. In the network settings tab click on *Edit* and select the following settings:
 - **VPC - required:** The one ending with (test-vpc)
 - **Subnet:** test-public-subnet
 - **Auto-assign Public IP:** Enable
5. Leave the rest as default and click on *Launch instance*.

Accessing the public EC2 instance

Try to access the public EC2 instance using the following command:

```
$ ssh -i ~/.ssh/aws-keypair ec2-user@<your-public-ec2-instance-public-ip>
```

You'll notice that the command hangs for a while and finally fails with a timeout error. Which means the EC2 instance didn't respond.

Observations

- The EC2 instance is not accessible from the internet.
- The reason is because the VPC and the subnet are private by default.
- Although we named one subnet as public, it is not public yet.

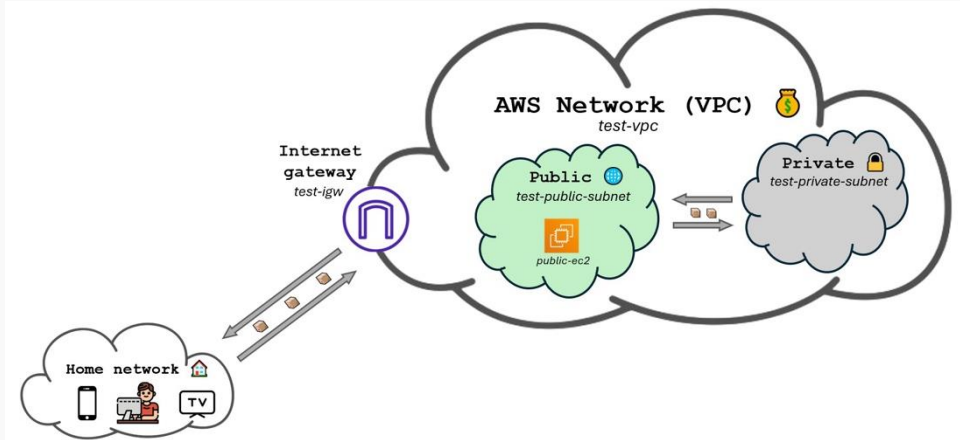
Solution

We need to create an **internet gateway** and a **routing table**.

- The internet gateway will be the entrypoint for the internet traffic to the VPC.
- The routing table will make internet traffic able to go from the internet gateway to a given subnet.
- Once the traffic can reach a subnet, it can reach the EC2 instances on that subnet.
- Doing this with an internet gateway means that the traffic is bidirectional. The EC2 instances can also reach the internet.

Accessing the public EC2 instance

To access the public instance, we'll first need to allow traffic to get to the VPC. We'll do this by creating an **Internet Gateway**.

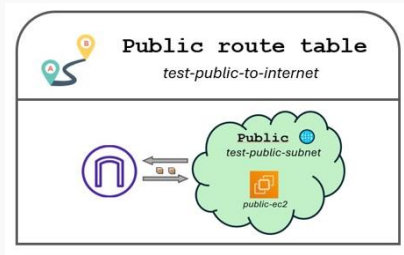


Creating an Internet Gateway

1. Go to the *VPC service*.
2. Click on *Internet Gateways* on the left navigation pane.
3. Click on *Create internet gateway*.
4. Fill the form with the following settings:
 - **Name:** test-igw
5. Click on *Create internet gateway*.
6. Now click on *Actions* on the top right corner and select *Attach to VPC*.
7. Choose the one ending with `test-vpc` and click on *Attach internet gateway*.

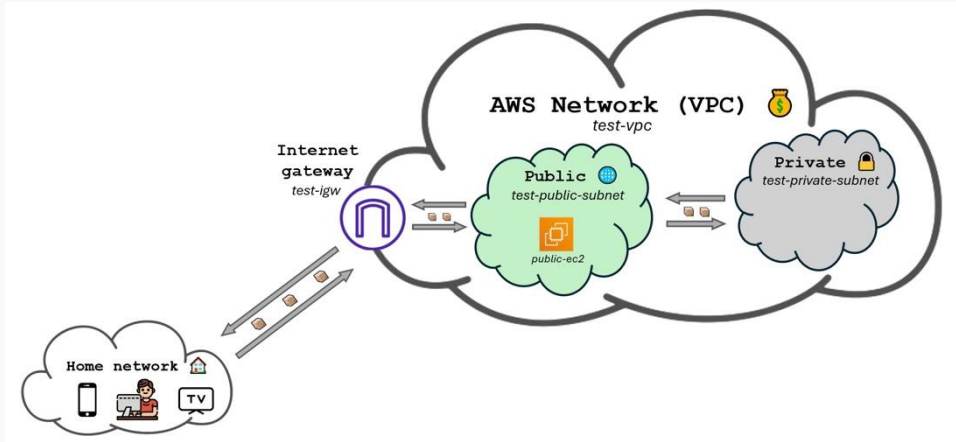
Accessing the public EC2 instance

Now traffic is getting to the VPC, but it still can't reach devices inside the public subnet. That is because despite its name, the public subnet is still private. We need to make it public by creating a **Route Table**.



Accessing the public EC2 instance

Once the table is created, traffic will be able to reach the public subnet.



Creating a Route Table

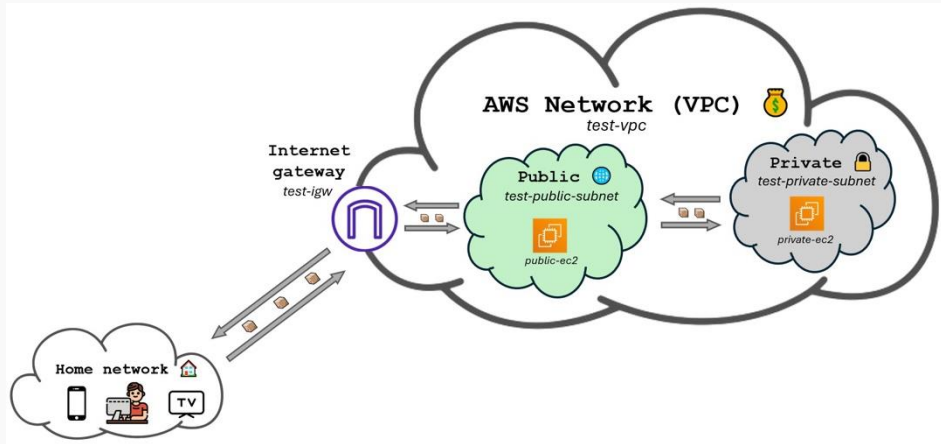
1. Go to the *VPC service*.
2. Click on *Route Tables* on the left navigation pane.
3. Click on *Create route table*.
4. Fill the form with the following settings:
 - **Name:** test-public-to-internet
 - **VPC:** The one ending with (test-vpc)
5. Click on *Create route table*.
6. Click on *Actions* on the top right corner and select *Edit subnet associations*.
7. Choose the public subnet and click on *Save associations*.
8. Click on *Actions* on the top right corner and select *Edit routes*.
9. Click on *Add route* and fill the form with the following settings:
 - **Destination:** 0.0.0.0/0
 - **Target:** Choose *Internet Gateway* and select the one ending with (test-igw).
10. Click on *Save changes*.

Accessing the public EC2 instance

- Now the EC2 instance is accessible from the internet.
- The public subnet is indeed public now.
- The private subnet is still private.

Deploying an EC2 instance in the private subnet

We are now going to launch an EC2 instance inside the private subnet.



Deploying an EC2 instance in the private subnet

1. Go to the *EC2 service*.
2. Click on *Launch instance*.
3. Fill the form with the following settings:
 - **Name:** private-ec2
 - **Key pair (login):** Choose `aws-keypair` (the one we have already been using)
4. In the network settings tab click on *Edit* and select the following settings:
 - **VPC - required:** The one ending with (test-vpc)
 - **Subnet:** test-private-subnet
 - **Auto-assign Public IP:** Enable
5. Leave the rest as default and click on *Launch instance*.

Accessing the private EC2 instance

We know we are not going to be able to SSH into the private EC2 instance since it resides in a private subnet. What we aim to do is to access the private EC2 instance from the public EC2 instance.

The first step is going to be to copy our private key onto the public EC2 instance so we can SSH into the private EC2 instance from there. Open a terminal in your **local machine** and run the following command:

```
$ scp -i ~/.ssh/aws-keypair ~/.ssh/aws-keypair ec2-user@<your-public-ec2-instance-public-ip>:/home/
```

This is just copying the private key to the public EC2 instance. Now we can SSH into the public EC2 instance and from there try to SSH into the private EC2 instance.

Accessing the private EC2 instance

SSH into the public EC2 instance using the following command:

```
$ ssh -i ~/.ssh/aws-keypair ec2-user@<your-public-ec2-instance-public-ip>
```

And run the following command to change the permissions of the private key:

```
[ec2-public]$ chmod 600 ~/.ssh/aws-keypair
```

From the public EC2 instance, try SSH into the private EC2 instance using the following command:

```
[ec2-public]$ ssh -i ~/.ssh/aws-keypair ec2-user@<your-private-ec2-instance-public-ip>
```

Accessing the private EC2 instance

You'll notice that the command hangs for a while and finally fails with a timeout error just as it did earlier when the public EC2 instance was not accessible from the internet. This is because we are trying to access the private EC2 instance using its **public** IP address. We need to use the **private** IP address instead.

Go to the *EC2 service* in the *AWS Management Console* and copy the **private** IP address of the private EC2 instance. Then SSH into the public EC2 instance and try to SSH into the private EC2 instance using the **private** IP address.

```
[ec2-private]$ ssh -i ~/.ssh/aws-keypair ec2-user@<your-private-ec2-instance-private-ip>
```

Test internet access from the private EC2 instance by running the following command:

```
[ec2-private]$ ping google.com
```

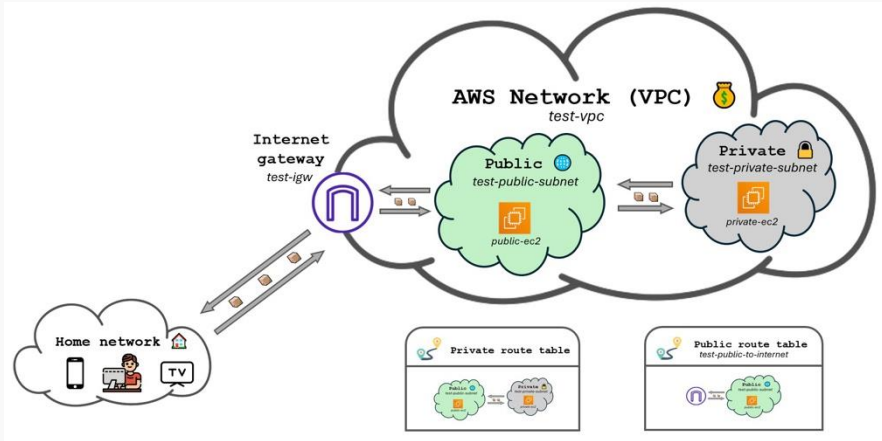
It should error out, signaling that there is no internet access.

Observations

- The EC2 instance is not accessible from the internet and will never be.
- We can access the private EC2 instance from the public EC2 instance.
- The private instance doesn't have access to the internet.

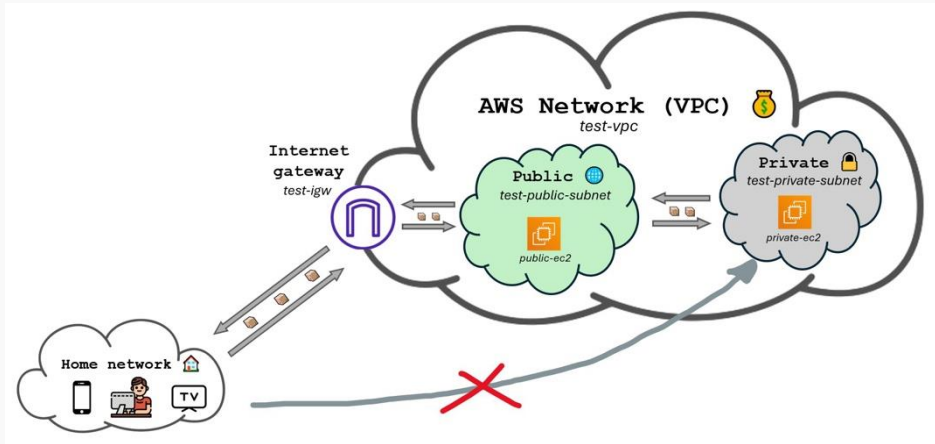
Recap

We ended up with this setup.



Recap

Where we can't directly access the private EC2 instance through SSH.



Recap

But we can access the private EC2 instance from the public EC2 instance.

